RAILWIZE

Technical interview preparation

156 questions. Practical answers.
From fundamentals to distributed-system trade-offs.

SPRING BOOT / ANGULAR / MYSQL
PAYMENTS / SECURITY / AI / TESTING

Includes worked examples, edge-case drills,
a demonstration plan and implementation references.

Based on the local Railwize source at commit 480484f.
Prepared 13 October 2026.

How to use this handbook

Begin with the opening explanation and architecture. Then study booking, payments, authentication and the journey planner: these are the strongest technical discussion areas.

For each answer, practice three layers: explain the concept in one sentence, connect it to Railwize, and describe one failure case or trade-off. Use the code index when asked where a feature is implemented.

The answers describe the project implementation and configurable defaults, not official Indian Railways policies. Illustrative fares and showcase train data are teaching examples. This is preparation material, not a new production certification or a promise that every interview question is covered.

Suggested preparation: Day 1 architecture and fundamentals; Day 2 booking, concurrency and payment; Day 3 security, AI and frontend; Day 4 practice the demonstration and difficult follow-ups.

Source basis: EVALUATION-GUIDE.md, backend/README.md, frontend/README.md and selected current service implementations. Planner candidate expansion and ranking are updated here to match the latest code. Repository-relative paths in the appendix can be located with IDE search.

Question map

Q1-20: Spring Boot, APIs, persistence and service boundaries.

Q21-47: Security, profiles, timetable rules and fares.

Q48-90: Journey alternatives, allocation, waitlist and payment recovery.

Q91-123: Events, emails, reporting and the Gemini assistant.

Q124-143: Angular, data integrity, testing and production trade-offs.

Q144-156: Latest planner fixes, support tickets, theme, collections, locking and interview follow-ups.

Contents

How to use this handbook 2

Contents 3

1. Your opening explanation 4

2. Understand the architecture before memorizing answers 5

3. Basic backend questions 6

4. Spring and distributed-system design questions 8

5. Authentication, sessions and access control 9

6. Catalogue, scheduling and fares 11

7. The distinguishing feature: journey alternatives 13

8. Booking, seat allocation and concurrency 15

9. Waitlist, cancellations and payment failures 17

10. Email, notifications, PDF and convenience features 20

11. Admin management, reports and exports 21

12. AI assistant: basic to difficult questions 23

13. Frontend and user experience questions 25

14. Persistence, reliability and testing 26

15. Edge-case drill: predict the result before demonstrating 28

16. A strong 15-20 minute demonstration 30

17. Practice these complete explanations 31

18. Difficult follow-up answers you should not improvise 33

19. How to answer an unfamiliar question 34

20. A focused revision plan 34

21. Implementation evidence index 35

22. Latest features and deeper technical follow-ups 39

1. Your opening explanation

30-second answer: What is your project?

I built Railwize, a railway reservation application using Angular and Spring Boot microservices. Passengers can search trains, book confirmed or waitlisted tickets, complete Razorpay test checkout, manage bookings and receive journey updates. Administrators manage the catalogue, users, bookings, reports and cancellations of specific dated train runs. My main distinguishing feature helps passengers discover a connecting journey or nearby-station alternative when an exact direct train is unsuitable.

Two-minute answer: Explain your project.

Railwize addresses the entire reservation lifecycle rather than only creating a booking record. The administrator configures stations, trains, class capacity, fare parameters and an ordered timetable. Passengers register and verify their email, or use Google sign-in. They search by boarding station, destination and date. Each matching train shows its configured classes with per-passenger fares and current availability. After choosing a class, they select 1-6 passengers on the booking page and receive a fresh group quote. The backend checks the train's operating day and timetable, calculates the distance-based fare and checks availability across the requested route segments. It creates a two-minute payment hold. A verified Razorpay test payment activates either a confirmed booking or a paid waitlist booking. Waitlisted bookings promote automatically when all passengers can be accommodated; otherwise they receive a full refund at the configured cutoff. The project also provides saved passengers, readable coach and berth descriptions, ticket PDFs, account-specific recent searches, calendar downloads, email and in-app updates. Administrators can cancel one dated train run, notify affected passengers and initiate full refunds without deducting the passenger cancellation fee. Its journey planner is my main differentiator. It searches one same-station interchange and optionally nearby origin and destination stations, checks transfer time and shows per-leg fares and availability. Each leg still has its own booking and PNR. A read-only Gemini assistant explains policies and retrieves authorized live booking information. I have also addressed concurrency, idempotency, asynchronous payment recovery, authorization and automated tests.

What must you say about scope?

It is an independently implemented academic reservation platform. It uses its own catalogue and seat inventory, and payments/refunds are Razorpay test-mode operations. It does not book into Indian Railways' live reservation system.

Do not present it as an official railway service, a completely production-hardened system or a proven replacement for every existing booking platform.

2. Understand the architecture before memorizing answers

```
Angular passenger/admin UI
          |
          v
API Gateway :8080
  |-- Auth :8081 -------- railway_auth_db
  |-- User :8082 -------- railway_user_db
  |-- Train :8083 ------- railway_train_db
  |-- Schedule :8084 ---- railway_schedule_db
  |-- Reservation :8085 - railway_reservation_db
  |       |-- PNR :8086 - railway_pnr_db
  |       |-- Payment :8087 ---- railway_payment_db ---- Razorpay TEST
  |       |-- Booking events --> RabbitMQ / internal HTTP
  |                                  |
  |                                  v
  |                         Notification :8088
  |                         railway_notification_db --> SMTP
  |-- Chatbot :8089 ---- Gemini + in-memory RAG/memory/cache

Config Server :8888 supplies centralized configuration.
Eureka :8761 supplies service registration/discovery.
```

There are **12 independently runnable Spring Boot services**, including infrastructure services, and **8 service-owned MySQL schemas**. Multiple schemas may share one local MySQL instance; database ownership does not imply eight physical database servers.

Component	Explain it in one sentence
Angular	Presents forms, pages and state; sends requests and renders backend results.
Gateway	Provides the browser-facing API entry point and routes requests to services.
Auth	Owns accounts, credentials, verification, sessions, roles and admin invitations.
User	Owns editable profile details and the passenger Master List.
Train	Owns stations, coordinates, train metadata, class capacity and fare parameters.
Schedule	Owns ordered stops, times, day offsets and cumulative route distances.
Reservation	Owns booking state, allocation, search/planning, cancellation, tickets and booking reports.
PNR	Generates and persists booking reference numbers.
Payment	Verifies test payments and tracks provider reconciliation/refunds.
Notification	Receives durable booking-email events and sends branded messages/attachments.
Chatbot	Combines policy retrieval with authorized read-only tools.
Eureka	Lets services register and locate one another by service identity.
Config Server	Supplies service configuration from the project's bundled configuration repository.

Important separation: Auth-service sends account emails. Notification-service handles booking lifecycle emails. In-app notification records belong to reservation-service. Therefore successful OTP email delivery alone does not prove the booking-email path works.

3. Basic backend questions

Q1. What is a backend?

The backend receives requests, validates data, applies business rules, accesses databases and integrates with providers. In Railwize, Angular can display a fare, but the backend decides the authoritative fare, ownership, seat allocation and payment state.

Q2. What is Spring Boot, and why use it?

Spring Boot simplifies creating Java applications through auto-configuration, dependency starters and an embedded server. Railwize combines Spring Security, Data JPA, validation, HTTP APIs and configuration without manually wiring every framework component. It reduces setup work; it does not automatically implement correct booking rules.

Q3. What is an API? What is REST?

An API is a contract between clients and a service. REST-style HTTP endpoints represent operations on resources. Examples include retrieving a timetable with GET, creating a booking with POST and updating a profile with PUT. JSON is the main data format. Correct status codes and predictable request/response structures make the contract usable.

Q4. Explain controller, service and repository.

The controller handles HTTP input/output; the service implements business rules; the repository accesses persistence. For scheduling, `ScheduleController` accepts the request, `ScheduleService` validates route chronology/catalogue consistency and the repository saves stops. Keeping these responsibilities separate makes tests and changes easier.

Q5. What is dependency injection?

Spring constructs collaborating objects and supplies their dependencies. A service receives a repository or client rather than constructing it with `new`. Constructor injection makes required dependencies explicit and allows tests to replace them with mocks.

Q6. Explain the main annotations.

Annotation	Purpose
`@SpringBootApplication`	Application entry configuration, component scanning and auto-configuration.
`@RestController`	HTTP controller whose return values become response bodies.
`@RequestMapping`, `@GetMapping`, `@PostMapping`	Map URLs and HTTP methods.
`@RequestBody`	Deserialize request JSON into a Java object.
`@PathVariable` / `@RequestParam`	Read URL path/query values.
`@Valid`, `@NotNull`, `@NotBlank`, `@Min`, `@Max`	Validate declared input constraints.
`@Service`	Register a business-layer component.
`@Entity`, `@Id`, `@Column`	Map persisted objects and column constraints.
`@Transactional`	Define a local database transaction boundary.
`@Scheduled`	Run background work at configured intervals.
`@RequiredArgsConstructor`	Lombok generates a constructor for required fields.
`@Slf4j`	Lombok provides an SLF4J logger.

Annotations do not replace cross-field validation. The relation between source/destination, chronology and cumulative distance needs business validation.

Q7. What is a DTO? Why not return the entity?

A DTO defines the API representation separately from the storage model. It prevents accidental exposure of internal fields, allows purpose-specific validation and reduces coupling to JPA relations. A ticket response can include readable seat metadata without changing the internal seat identifier.

Q8. What are JPA and Hibernate?

JPA is the persistence specification; Hibernate is an implementation. Spring Data JPA provides repository abstractions. SQL still matters for constraints, indexes, transactions and locks. ORM does not remove the need to understand the database.

Q9. What is Maven?

Maven manages Java dependencies and build/test lifecycle through `pom.xml`. Each backend service has its own Maven project. This repository does not have a root Maven reactor; run Maven inside a service or use the backend verification helper.

Q10. What is the difference between an exception and an HTTP error?

An exception is an internal Java failure signal. An exception handler translates expected failures into appropriate API responses. Invalid data may return 400, a state conflict 409 and an unavailable dependency 503. The response should not expose a stack trace or secrets.

4. Spring and distributed-system design questions

Q11. Why microservices instead of a monolith?

I wanted clear ownership of identity, catalogue, scheduling, bookings and payments, and experience with service discovery, internal APIs and asynchronous processing. The cost is more deployment, debugging and consistency complexity. For a small production team, a modular monolith could be simpler; microservices are a deliberate learning/design choice, not automatically faster or better.

Q12. Why separate auth-service and user-service?

Identity and sessions change for different reasons than personal profile/passenger data. Auth-service is authoritative for verified email, account status and role. User-service stores profile details and saved travellers. Stable account identity keeps ownership intact when a public username changes.

Q13. Why separate train and schedule?

Train-service defines what a service is and how classes are priced/configured. Schedule-service defines where and when it stops. Reservation-service combines them to determine bookability and fare. Splitting them adds calls and failure modes, so their contracts need validation and tests.

Q14. Why have a gateway? Is it the only security layer?

It provides a single API origin and routing/CORS/documentation integration. It is not the only security layer. Resource services also verify authentication, role and ownership. Calling a service directly must not bypass authorization.

Q15. What is Eureka? Does it load configuration or store bookings?

Eureka is a registry for service discovery. Config Server supplies configuration. MySQL stores domain data. These are different responsibilities.

Q16. What is OpenFeign? Is all inter-service communication Feign?

OpenFeign exposes an HTTP client through a Java interface. Several services use it to call catalogue, schedule and other internal APIs. Communication is not exclusively Feign: the reactive chatbot also uses WebClient, and messaging/email event handoff uses its own transports.

Q17. What are synchronous and asynchronous operations here?

Search/quote and validation need request-response results. Emails and refund reconciliation continue asynchronously after a state change. Asynchronous work improves separation and recovery, but requires durable state, retries and clear pending statuses.

Q18. Does `@Transactional` roll back a Razorpay payment?

No. A local transaction covers that service's database, not another database, RabbitMQ, SMTP or Razorpay. The project uses idempotent operations, durable events, reconciliation and compensating refunds across boundaries. It is not a distributed ACID transaction or a dedicated saga-engine implementation.

Q19. What is a circuit breaker?

Closed permits calls while recording outcomes. Open temporarily rejects calls after enough failures. Half-open permits limited probes to determine recovery. Railwize uses Resilience4j for Gemini and the PNR dependency. It does not wrap every remote call in a breaker. Timeout limits call duration; a breaker limits repeated calls to an unhealthy dependency.

Q20. What is a safe fallback?

For AI, a clear unavailable response or safe policy fallback can be useful. For PNR allocation, fabricating a reference would be unsafe; the caller receives an unavailable error. A fallback must preserve business correctness rather than merely return HTTP 200.

5. Authentication, sessions and access control

Q21. Explain registration.

Registration creates a passenger account and sends a verification OTP. The OTP is purpose-scoped, hashed, time-limited and attempt-limited. Successful verification activates the account, sends a welcome email and directs the Angular user to sign in. Public registration never accepts an ADMIN role choice.

Q22. Are passwords encrypted?

> They are BCrypt-hashed, not reversibly encrypted. Password verification compares the submitted password using BCrypt. I cannot decrypt a stored password; password reset replaces its hash after verification.

Hashing passwords is different from signing JWTs or hashing idempotency request fingerprints.

Q23. What is a JWT?

A JWT carries signed claims such as identity/session information. A signature detects unauthorized modification; it does not hide the payload. Passwords and provider secrets do not belong in it. Railwize also checks server-side session/account state, so it is not a completely stateless authentication design.

Q24. Why both access and refresh tokens?

Access tokens authorize API requests; refresh credentials obtain a new access token through the controlled session flow. Logout/session invalidation must revoke the server-side session, not just delete browser state. The API client coordinates refresh and avoids uncontrolled retry loops.

Q25. How does logout invalidate an already issued JWT?

Protected services check the token's session/account state with auth-service. Revoking that session means the JWT is no longer accepted even if its signature and timestamp are otherwise valid. This gives immediate revocation at the cost of an auth-service dependency and additional requests. Auth unavailability fails closed.

Q26. Authentication versus authorization?

Authentication establishes who you are. Authorization decides what you can do. Passenger A may be authenticated but still forbidden from downloading passenger B's ticket. ADMIN permissions and owner checks are enforced in the backend, not inferred from a visible frontend button.

Q27. How do you prevent someone registering as admin?

The first admin uses explicitly enabled bootstrap configuration; it is disabled afterward. Additional admins receive expiring, single-use invitations tied to a matching verified email. Acceptance revokes old sessions, requiring fresh login with the new role. There is no public role dropdown or shared default admin password.

Q28. Why more than one administrator?

It supports operational continuity and separate accountable accounts instead of sharing credentials. The project has ADMIN and passenger roles; it does not implement a full hierarchy of finance/operator/support permissions. More granular roles are a future extension.

Q29. How does Google login work?

The backend initiates the browser flow, checks state/nonce and Google identity, then redirects to Angular with a short-lived single-use exchange code. Angular exchanges it in the same browser session for application credentials. Application access/refresh tokens are not placed in the redirect URL. Google authenticates the identity; Railwize still controls its own roles and sessions.

Q30. What caused `redirect_uri_mismatch`?

The callback sent to Google must exactly match a registered authorized redirect URI. Locally, Google's callback is `http://localhost:8080/api/auth/google/callback`; Angular completion is separately `http://localhost:4200/auth/callback`. Ports, scheme, hostname and path matter. `localhost` and `127.0.0.1` are not interchangeable OAuth origins.

Q31. How did you fix missing `/me` profile data?

An existing verified account may not yet have a user-service profile row. GET now returns an empty profile view with authenticated account information instead of treating it as an unknown account. PUT creates or updates editable details. GET does not silently insert a profile row.

Q32. Why allow only one username change?

It lets a Google user personalize a generated username while making public identity easier to follow. The stable internal account key continues to own bookings, notifications and saved data. The single-change rule is a product decision, not a technical necessity.

Q33. Where are frontend tokens stored? Is that completely secure?

They are stored in sessionStorage, cleared on logout/session failure. Unlike persistent localStorage, this limits their browser lifetime, but JavaScript can still access them: XSS remains a concern. Secure HttpOnly cookie/BFF designs are a possible future improvement with corresponding CSRF and deployment considerations.

Q34. Does a Swagger padlock prove that I am logged in?

No. Swagger may only have a locally stored token. The backend decides whether a protected request is authorized. Shared gateway Swagger remembers within one gateway run and clears stored authorization after gateway restart. This UI behavior does not revoke every server session. Swagger's dialog Logout and the backend logout API are different operations.

Q35. Why do login/register pages redirect logged-in users?

Guest-route guards avoid presenting redundant account forms. Admins land in `/admin`; passengers go to their appropriate journey/account flow. Guards improve navigation but are not a security boundary.

Q36. Does blocking a user cancel their tickets?

No. Blocking revokes sessions and prevents account access; it does not erase obligations, cancel existing bookings or stop pending refunds. Eligible passenger accounts can be reactivated. Self/admin and certain inactive/deleted accounts are protected from this action.

6. Catalogue, scheduling and fares

Q37. Why is distance not entered as a single value on a station?

A station does not have one universal distance. Distance is cumulative along a particular train's route, recorded on timetable stops. A train may reach the same station over a different route. Fare uses destination cumulative distance minus boarding cumulative distance.

Q38. Explain fare calculation with numbers.

Assume an illustrative class has base fare INR 100, rate INR 0.50/km and minimum fare INR 150. Stops A, B and C have cumulative distances 0, 200 and 500 km. B -> C is 300 km. Per-passenger fare is `max(150, 100 + 0.50 x 300) = INR 250`. For two passengers the total is INR 500. These are teaching values, not an official railway tariff.

Q39. Why BigDecimal rather than double for fares?

Binary floating-point cannot exactly represent many decimal currency values. BigDecimal supports explicit decimal precision and rounding. The project rounds each passenger fare to two decimal places and stores a booking price snapshot.

Q40. What if someone changes the fare in browser tools?

The server recalculates the quote and compares the expected total with the authoritative amount. Payment orders use saved server amounts. A mismatched fare produces a conflict instead of a cheaper booking. The UI is never trusted to decide the payable amount.

Q41. Why null source arrival and destination departure?

For that published route, the first stop has no preceding arrival and the last stop has no onward departure. Angular disables those fields and normalizes them to null. Backend validation independently enforces the rule. Disabling an input alone would not protect the API.

Q42. How do overnight schedules work?

Each stop has an arrival day and optional separate departure day. Arrival 23:55 on Day 1 followed by departure 00:05 on Day 2 is valid. Comparing clock times alone would incorrectly reject it; chronology is checked with day offsets.

Q43. Boarding date versus source run date?

Suppose a train originates on Monday and reaches a passenger's boarding stop on Day 2. Tuesday is that passenger's boarding date, but Monday identifies the run. `runDate = boardingDate - (departureDayNumber - 1)`. Operating weekdays, inventory and dated operator cancellation must use this source run identity.

Q44. Why can a train exist but not appear in search?

It may lack a published timetable, class configuration, valid fare or route stop; the station order may be reversed; the calculated source weekday may be inactive; today's departure may have passed; the date may exceed two months; or the run may be cancelled. A train row alone is not a bookable journey. Dependency errors should also be distinguished from a legitimate empty result.

Q45. Why prohibit changing published timetables?

Existing bookings depend on stop sequence, distance, day offsets, inventory keys and departure times. Arbitrary edits could invalidate them. The implementation rejects a different replacement while accepting an identical submission. A future version could use versioned schedules and explicit operational migration rules.

Q46. Is the advance window 60 days?

No. It is today through two calendar months ahead, inclusive, in Asia/Kolkata. Calendar month lengths differ. Today's date alone is insufficient: the selected boarding departure must still be in the future.

Q47. What latitude/longitude do you enter?

The station's physical location in decimal degrees: latitude -90 to 90, longitude -180 to 180, supplied together. Indian locations generally have positive north/east values. Coordinates are used for geographic proximity, not the route-kilometre fare calculation. Incorrect coordinates produce misleading nearby suggestions even if values pass range checks.

7. The distinguishing feature: journey alternatives

Q48. What real-world problem are you solving?

When the exact station pair has no suitable direct train, a passenger has to check alternate terminals, find an interchange and compare times manually. Railwize combines these checks. It searches a one-interchange journey and nearby-station alternatives, then presents actual stations, dates, transfer wait, class availability and fares in one place.

Avoid claiming that no other real-world product offers a similar feature. Explain the problem and your working implementation rather than making an unverified exclusivity claim.

Q49. Give a complete connection example.

The fictional showcase service RW901 leaves NZM at 06:00 and reaches LJN at 14:30. RW902 leaves LJN at 16:00 and reaches BSB at 21:00. The 90-minute interchange satisfies a 60-360-minute transfer window. The planner can suggest this for NZM -> BSB in a configured common class. It rejects the showcase 14:50 departure as too tight and a 23:45 departure as beyond the default maximum wait.

Q50. Give a nearby-station example.

For CSMT -> ADI, the fictional RW905 service runs BDTS -> SBT. With suitable coordinates and radius, the planner shows Bandra Terminus and Sabarmati as the actual departure/arrival stations. It does not relabel the ticket as CSMT -> ADI. The passenger acknowledges alternate stations and handles local transport separately.

Q51. Explain the algorithm.

1. Validate stations, date, class, count and transfer/radius parameters.

2. Include exact endpoint stations; optionally add all located stations within the selected radius at both ends, subject to catalogue and work limits.

3. Load bounded eligible catalogue routes and index routes by departure-capable stops.

4. Inspect forward first-leg segments and compatible second trains at the same interchange station.

5. Calculate first arrival and the earliest/latest acceptable second departure, including midnight rollover.

6. Validate both legs using the existing quote rules, operating days and cancelled-run checks.

7. Reject loops, invalid direction, unsuitable timing and unavailable journeys.

8. Deduplicate and rank confirmed-capable plans first, then fewer legs, lower combined endpoint distance, earlier final arrival, lower total fare and a stable ID tie-breaker. Nearby direct candidates are examined before connections.

9. Return at most 20 options and notices when search limits constrain results.

This is bounded timetable-based enumeration, not an implemented Dijkstra, A*, neural network or unrestricted journey graph solver.

Q52. What is Haversine?

It estimates great-circle distance between latitude/longitude points on a sphere. The implementation uses an Earth radius around 6,371 km and filters stations by the selected radius. It is not road distance, travel time, platform walking time or fare distance.

Q53. What are the search limits?

One same-station interchange, radius 1-100 km, minimum interchange 30-240 minutes and maximum up to 720 minutes. Defaults are all classes, 50 km and a 60-360-minute transfer window. Each connection uses a class offered on both legs. All located stations within the radius are candidates at both ends. Nearby direct routes are examined before connections. The implementation scans at most 10,000 trains/stations within a 12-second work budget and other candidate limits, and returns at most 20 plans with partial-result notices. The Angular controls expose a smaller set of choices. Bounded results are not a guarantee of the globally optimal itinerary.

Q54. Why not give one ticket/payment for the entire connection?

That requires coordinating two separate inventories and resolving partial success: leg one might be paid while leg two becomes unavailable. It also needs refund/compensation, connection protection and cancellation policy across legs. The current feature solves discovery while reusing independently verified booking/payment flows. Each leg has its own PNR and payment.

Q55. Does the algorithm guarantee that the connection will be caught?

No. It validates scheduled transfer time, not live delays or platform movement. A delayed first train can still cause a missed connection. If one leg is operator-cancelled, an independently booked onward leg is not automatically cancelled or refunded under the train-run action.

Q56. Why not use machine learning? Can the process be automated?

The implemented planner is already automated: it computes options from timetables and constraints. ML is unnecessary for a deterministic rule such as "departure must be at least 60 minutes after arrival." ML could later rank feasible routes by preferences or estimate missed-connection risk using historical delays. Hard safety/date/inventory rules would remain deterministic.

Q57. What data and evaluation would an ML extension need?

Historical scheduled/actual arrivals, cancellations, transfer times, station conditions and user feedback. Train/validate chronologically to avoid future information leakage. Compare against a timetable-only baseline using route-selection quality, calibration of delay risk and missed-connection outcomes. Without reliable data, claiming a trained prediction model would be misleading. No custom route-prediction model is currently trained here.

Q58. What is the computational complexity?

The planner examines combinations of candidate origins, first-route segments and matching second routes/stops. Cost grows with catalogue size and route length, plus remote calls; it is not simply O(n). Station indexing, request-local quote reuse and explicit limits constrain work. For scale, precomputed adjacency, spatial indexes, timetable caches and specialized routing algorithms would be preferable. Do not invent benchmark numbers.

8. Booking, seat allocation and concurrency

Q59. Explain one booking from click to ticket.

1. Angular obtains a fresh server quote for the train, stations, boarding date, class and passenger count.

2. The passenger enters details or selects saved travellers.

3. Angular submits the expected fare and an idempotency key.

4. The backend validates input, recalculates the fare and obtains a PNR.

5. Inside a transaction, it locks the run and class inventory, checks cancellation/expiry and allocates all requested seats or a WL checkout.

6. It saves `PENDING_PAYMENT` with the server deadline, normally two minutes.

7. Payment-service creates a Razorpay test order from the saved amount.

8. Checkout completes; backend verification establishes that the payment is captured and belongs to the expected order.

9. Reservation-service safely transitions to `CONFIRMED` or `WAITING`.

10. Durable events drive email/PDF delivery and in-app notifications. Reconciliation handles interrupted callbacks and later refunds.

Q60. Explain the state machine.

```
PENDING_PAYMENT -- verified captured payment + held seats --> CONFIRMED
PENDING_PAYMENT -- verified captured payment + WL path ----> WAITING
PENDING_PAYMENT -- hold deadline -------------------------> EXPIRED
PENDING_PAYMENT -- cancellation --------------------------> CANCELLED
WAITING --------- complete allocation before cutoff ------> CONFIRMED
WAITING --------- cancellation or cutoff -----------------> CANCELLED
CONFIRMED ------- passenger/operator cancellation --------> CANCELLED

Late capture after expiry/cancellation: refund path, not resurrection.
```

CNF and WL are user-facing labels. Backend booking enums include `CONFIRMED` and `WAITING`. Payment and refund states are separate dimensions: "cancelled" does not mean "refund processed."

Q61. Why two-minute holds?

They give time to complete checkout while limiting inventory blocked by abandoned sessions. The server owns the deadline; changing browser time does not extend it. A hold can be shorter near departure or the WL cutoff. A CNF checkout holds seats; an unpaid WL checkout has neither seats nor a paid queue place.

Q62. What happens if two users request the last seat simultaneously?

Both may see available capacity in a read-only search. Final allocation uses database pessimistic locks and rechecks availability inside the transaction. The first successful allocation consumes the position; the later request observes the updated state and cannot allocate the overlapping seat again. It may receive WL if eligible. Search is a snapshot, not a reservation.

Q63. Exactly what is locked?

The booking engine acquires a train/source-date run lock, then its class-inventory lock; transitions also lock/refresh relevant reservations. `SeatInventoryRepository.lock` uses `PESSIMISTIC_WRITE`. A consistent order coordinates creation, payment acceptance, expiry, promotion and whole-run cancellation. It serializes work for the run, which improves correctness but limits throughput on a highly contested departure.

Q64. Is `synchronized` enough? Is `@Transactional` alone enough?

No. Java synchronization only coordinates threads in one process. Different service instances share a database, so allocation needs database coordination. A transaction alone does not necessarily stop two readers making the same decision; the lock strategy, queries and constraints matter. Actual MySQL load/concurrency testing remains important beyond isolated H2 tests.

Q65. How does route-segment seat reuse work?

For route A -> B -> C -> D, a berth used A -> B can be reused B -> D. A berth used A -> C cannot simultaneously be sold B -> D. Model segments as half-open intervals `[fromSequence, toSequence)`: overlap exists when `existingFrom < requestedTo` and `existingTo > requestedFrom`. Allocation considers occupied seat identifiers for overlapping active journeys.

Q66. Why not store a single available-seat count per train?

It cannot represent seats becoming free at intermediate stations, and a count alone does not prove the same physical berth is free across the entire requested journey. The implementation checks seat identifiers across overlaps, ensuring a passenger does not change seats mid-journey merely because segment totals looked sufficient.

Q67. What happens on a double click or a network retry?

The booking request has an account-scoped idempotency key and a SHA-256 fingerprint of the relevant payload. The same key and same request return the existing booking; reusing the key for different details causes a conflict. Disabling the button is useful UI behavior but is not the guarantee.

Q68. Can an idempotency retry create an unused PNR?

Concurrent attempts can obtain separate references before one wins the reservation write; replay still ensures one logical booking. Unused PNRs are preferable to duplicate reservations, though more tightly coordinated allocation could reduce reference gaps. Idempotency does not require consecutive PNR numbering.

Q69. How are PNRs generated, and is the code perfect?

The service generates a random ten-digit value, checks whether it exists and saves it in a unique database column. The unique constraint is the final protection. The preliminary existence check is not atomic: two concurrent requests could choose the same unused value and one insert could fail. A bounded retry specifically for unique-key collisions or another collision-resistant allocation strategy would improve this. Do not repeat the source comment claiming the existence loop alone guarantees concurrency-safe uniqueness. A PNR is a reference, not a password; ownership authorization protects access.

Q70. How are coach and berth labels generated?

Available internal identifiers are enumerated by configured coach/position order, such as `SL-C1-7`. `SeatLayouts` converts that to a readable coach/berth description, such as S1, berth 7, side lower. This is a display enhancement over the allocation engine, not a new capacity system.

Class	Prefix	Detailed layout
SL	S	Eight-position three-tier pattern.
3A	B	Eight-position three-tier pattern.
2A	A	Six-position two-tier pattern.
1A	H	Numbered berth; cabin/coupe type is not guessed.
3E	M	Numbered berth; detailed variant is not guessed.
EC	E	Numbered seat.
2S	D	Numbered seat.
UR	GS	Project's configured numbered-seat representation.

The simulated layouts do not claim universal compatibility with every actual coach type or real-world unreserved travel policy.

Q71. Explain the berth formula.

For SL/3A, `(position - 1) % 8` indexes: lower, middle, upper, lower, middle, upper, side lower, side upper. Thus 1 is lower, 7 side lower, 8 side upper and 9 begins the next block. For 2A the six-position pattern is lower, upper, lower, upper, side lower, side upper.

Q72. Does choosing LOWER guarantee a lower berth?

No. The project validates/stores the preference but currently allocates from the available ordered positions rather than optimizing preferences. Do not claim age-based, gender-based, family-adjacency or accessibility-priority allocation. Those require additional policy and algorithm work.

9. Waitlist, cancellations and payment failures

Q73. Why remove RAC and mixed passenger status?

One booking collects one payment for up to six passengers. Keeping the entire group CNF or WL simplifies allocation, cancellations, refund amounts and understandable ticket state. RAC and partial-group confirmation require finer passenger-level inventory and financial rules. This implementation deliberately omits them.

Q74. Explain paid waitlist promotion using numbers.

Assume a class has two positions, both booked. Booking A with two passengers completes WL payment; later booking B with one passenger also pays for WL. If one position is released, neither bypasses A: A must fit in full. When both positions become available, A confirms automatically without another payment, gets assigned seats and receives an updated email/PDF/in-app notice.

Q75. What exactly is FIFO here?

Among eligible paid WL bookings in one train source-date/class, priority follows increasing booking creation ID across route segments. It is not passenger-by-passenger priority and is not strictly payment-timestamp ordering. The head booking must fit in full; later smaller bookings cannot jump it. The trade-off is head-of-line blocking and potentially unused capacity.

Q76. What is the WL cutoff?

By default one hour before the booking's boarding departure. Remaining paid WL bookings are cancelled and fully refunded. Expiry precedes promotion, including at the exact cutoff boundary. Lifecycle maintenance runs on relevant operations and in a scheduled scan, normally every ten seconds; this is not guaranteed zero-latency real-time processing.

Q77. What if the user cancels a INR 1,000 ticket?

For a paid confirmed booking under the default 20% rule, the fee is INR 200 and requested refund INR 800. Paid WL cancellation requests INR 1,000 with no deduction. An unpaid booking has no money to refund. These are Railwize's configured rules, not the complete Indian Railways refund tariff.

Q78. Can a user cancel only one passenger?

No. Cancellation applies to the whole booking. This matches whole-group confirmation and one-payment accounting. Partial cancellation is a possible future enhancement, requiring passenger-level refund and allocation rules.

Q79. Why show a cancellation preview?

It explains the current deduction and expected refund before mutation. It is still a snapshot: a WL ticket may promote between preview and cancellation. The final action rechecks current state; the frontend must refresh a stale estimate rather than silently treating the old preview as permanent.

Q80. What is different about operator train cancellation?

It targets train number plus the original source departure date, not the train entity. The ADMIN previews impact, supplies a reason and confirms the action before source departure. Active bookings across classes and boarding segments are cancelled; captured fares receive a full refund without the passenger cancellation fee. Other run dates remain bookable and audit/history records are preserved.

Q81. What if booking and operator cancellation happen together?

Both coordinate through the run lock. If cancellation wins, final booking allocation rejects the run. If a booking commits first, cancellation includes the active booking. Payment acceptance also checks cancelled state; late captures follow full-refund processing. Removing the run only from search would be insufficient because a user may hold an older quote.

Q82. Can a cancelled run be restored or cancelled after it starts?

Not in the current implementation. Cancellation is permanent for that date and is permitted before the first station's departure. Partial-route cancellation, diversion, reinstatement and disruption after source departure need different policies. Already cancelled/expired bookings are not reprocessed, and previous passenger-cancellation fees are not retroactively recalculated by this action.

Q83. What is the difference between delete train and cancel train run?

Deleting the catalogue entry risks breaking historical references and affects every date. Cancelling a dated run records an operational event, prevents that run's new bookings and preserves evidence for tickets and refunds. This is why the feature is dated cancellation rather than indiscriminate deletion.

Q84. What proves a payment succeeded?

Not a browser success message. Payment-service validates the checkout signature, verifies provider payment/order state and checks the server-owned amount/association before notifying reservation-service. Webhooks also require the configured verification mechanism. All provider operations are restricted to test mode.

Q85. Why keep booking status separate from payment status?

A paid booking may still be WL. A cancelled booking may have a pending refund. A late successful capture may belong to an expired booking and require refund rather than confirmation. One `success` boolean cannot describe these states safely.

Q86. What if the browser closes after payment?

Browser callback delivery is not guaranteed. Provider webhooks and scheduled reconciliation provide independent recovery paths. The user can revisit the booking to see current server state. A local webhook cannot be reached by Razorpay without a reachable endpoint; provider polling/reconciliation still matters.

Q87. What if payment is captured after the two-minute hold?

The expired booking is not revived because its seats may have been reassigned. The captured amount goes through a full-refund path. Confirmation after payment is conditional on valid reservation state, not merely the existence of money at the provider.

Q88. What if a callback or webhook arrives twice?

State transitions, payment identifiers and deduplication prevent double confirmation/refund/event effects. A retry should converge on the existing result. Exactly-once delivery is not assumed. Reconciliation also handles missing or out-of-order provider information.

Q89. What if Razorpay goes down?

New checkout can become unavailable, holds may expire and pending reconciliation/refunds remain pending or failed awaiting further processing. The application must not invent payment success or mark a refund credited without confirmation. The UI keeps booking and refund progress distinct.

Q90. Did you process real refunds?

No real money is collected or refunded. I integrated Razorpay test orders, verification and refund-state processing so I can demonstrate the financial lifecycle safely. A production deployment would require live-provider onboarding, operational controls and additional validation.

10. Email, notifications, PDF and convenience features

Q91. What is an outbox, and why use it?

When booking state changes, the same local transaction stores an event to deliver later. A worker reads the durable outbox and hands it to notification-service through RabbitMQ, with authenticated HTTP fallback. A crash after database commit does not silently lose the intent to send the booking email.

Q92. What is an inbox? Can emails duplicate?

Notification-service persistently records incoming event IDs and delivery attempts, so repeated event handoffs do not normally resend completed deliveries. However, a crash after SMTP accepted mail but before completion was recorded can still cause a duplicate. The honest guarantee is durable retry/deduplication with a residual SMTP ambiguity window, not exactly-once email delivery.

Q93. Which emails are sent?

Account OTP, welcome, password-reset instructions and admin invitation use consistent branding. Booking events cover verified payment/booking information, WL promotion, failed/incomplete checkout, passenger or operator cancellation, WL expiry and refund progress. Confirmed/payment and promotion messages include ticket PDF attachments. A paid WL confirmation message must still clearly say WL; it is not a confirmed-seat promise.

Q94. Why can OTP arrive while booking email fails?

They use different paths. Inspect the reservation event outbox, RabbitMQ/internal HTTP handoff, notification inbox/retry state, attachment generation and SMTP errors. Successful auth-service SMTP does not prove that the booking event reached notification-service. An email failure does not roll back an already committed booking.

Q95. Are in-app notifications WebSockets?

No. The frontend polls the authenticated inbox every five seconds while active/visible and refreshes after relevant actions. The backend stores unread/read state separately from email delivery. Polling is simpler but creates request load and is not literally instant push; SSE/WebSockets could be a later enhancement.

Q96. How do PDFs work? Is the PDF a database record?

PDFBox renders the authorized ticket response with branding, passenger/seat information, fare/refund state, timestamps and page information. Bundled fonts make it portable. Downloaded PDFs reflect state at generation time; previously saved PDFs do not update after cancellation. Backend ownership and payment checks protect downloads.

Q97. Does a PDF or PNR alone prove a ticket is still valid?

No. A file can be old or copied. Current backend status is authoritative. This is also a test inventory system, not a live railway-issued ticket. There is no implemented official conductor verification or anti-forgery system merely because the document looks professional.

Q98. What is the passenger Master List?

It stores owner-scoped traveller details for selection during booking. Booking copies a snapshot, so later editing a saved passenger does not rewrite an old ticket. Saved age can become stale and should be reviewed. The API retains `birthPreference` for booking and uses `berthPreference` for saved passengers; Angular maps this naming difference.

Q99. What changed in recent searches?

The last five valid searches are scoped to the immutable signed-in account in localStorage. Guest history is in memory. Account switching clears visible state and loads only the new account's history. Restoring a search triggers fresh availability; stored searches are not cached seat guarantees. Account scoping is not encryption against someone controlling the browser profile.

Q100. How does Add to calendar work?

Angular generates an `.ics` file for a future confirmed departure using the server timestamp, including a one-hour reminder. The user imports it into a calendar app. It is not a Google Calendar API synchronization: cancellation does not automatically delete an imported event, and the app should not invent an arrival time absent from the ticket response.

Q101. What is the Help section?

A public, searchable, categorized set of guidance for accounts, bookings, payments, waitlists, refunds and operational cancellation, with relevant navigation links. It remains useful when the AI provider is unavailable. The Help centre now links to a support-case workflow: passengers create private requests with an optional owned PNR, administrators reply and track Open/In progress/Resolved status, and updates arrive through email and in-app notifications. It is not a synchronous live chat or a guaranteed response-time service.

11. Admin management, reports and exports

Q102. Summarize everything an administrator can do.

View dashboard metrics/popular trains; maintain stations, trains and timetables within publication rules; inspect/search/filter users and booking histories; block/reactivate eligible passenger accounts; inspect/cancel bookings; review financial/cancellation reports; export CSV; invite administrators; and preview/cancel dated train runs with audit/refund progress. Personal account and help features remain relevant to admins as well.

Q103. How do you define popular trains?

The top ten trains by verified paid booking/PNR count, including paid WL and bookings later cancelled. It is not passengers carried, current confirmed passengers, occupancy percentage or gross ticket value. Unpaid holds do not raise the ranking. Always state the metric's definition.

Q104. What exactly is revenue in the dashboard?

It is a test-booking financial summary. Captured booking fares are verified associated paid fares, including paid WL. The net-after-all-refunds metric subtracts processed and outstanding refunds; outstanding includes failed refunds awaiting action. These figures are not recognized accounting revenue, profit, bank settlement or a complete payment-provider ledger.

Q105. Explain it with a numerical report.

Suppose captured booking fares total INR 10,000. Processed refunds total INR 2,000 and outstanding refunds INR 1,000. Net after all refunds is INR 7,000. Ignoring pending refunds would overstate retained fare. Provider settlement fees, tax recognition and business operating costs are not modeled as a complete profit calculation.

Q106. Is a monthly report based on payment month or journey month?

Journey date. A ticket paid in October for a November trip belongs to the November journey group. Date filters are inclusive and limited to supported ranges, with day/month/train grouping. A different accounting question needs a separate transaction-date report, not relabeling this result.

Q107. What is cancellation rate?

Cancelled PNR count divided by all PNRs in the selected group, including unpaid/expired records. It is a booking-based operational metric, not a passenger cancellation rate or paid-only cancellation rate. Empty periods are omitted rather than filled with invented activity.

Q108. Why pagination and sorting?

Returning every record repeatedly becomes slow and memory-heavy. Standard paged responses expose content, page, size and totals, with bounded size and allowed sort fields. Booking history also filters status and journey dates. Unsupported fields/ranges are rejected. At very high volume, keyset pagination and specialized reporting storage could improve performance.

Q109. Is export really Excel? What is CSV injection?

The implemented format is UTF-8 CSV with BOM, quoting and formula neutralization, readable by Excel. It is not native `.xlsx`. Spreadsheet software can interpret user-supplied fields starting like formulas; exports neutralize dangerous values. More than 10,000 matching rows requires narrower filtering rather than silently producing an incomplete report.

12. AI assistant: basic to difficult questions

Q110. What does the chatbot actually do?

It answers platform-policy questions, searches trains and retrieves the authenticated account's bookings, cancellations, counts, next journey or owned PNR details. It opens as a right-side panel and can expand to `/assistant`. Train cards enter the normal booking page, where passenger count is explicitly selected and the group is requoted. The assistant cannot independently book, cancel, charge, refund or change accounts.

Q111. What is RAG? Why not just ask Gemini?

Retrieval-Augmented Generation retrieves relevant project knowledge and places it in the model context. Railwize embeds sections of its FAQ into SimpleVectorStore so answers can use its actual two-minute hold, cancellation and waitlist rules. Generic model knowledge may describe a different platform or outdated rules. RAG improves grounding but does not guarantee truth.

Q112. What is an embedding/vector store?

An embedding is a numerical representation used to compare semantic similarity. The vector store finds FAQ sections related to the question even when wording differs. It is separate from MySQL transaction records. SimpleVectorStore is in memory here and must be rebuilt/reloaded on service restart.

Q113. RAG versus function calling?

RAG supplies relatively static policy text. Tools query live authorized service data such as a current booking or seat quote. Availability must not be answered from an old FAQ or invented model text. Some common account questions use a deterministic service-backed path without consuming an LLM call.

Q114. Where is conversation memory stored?

MessageWindowChatMemory uses InMemoryChatMemoryRepository, keeping a recent 20-message window. Conversations are owner-checked, bounded and have a 24-hour lifetime. They reset with service restart; this is not unlimited permanent memory or a shared multi-instance conversation database.

Q115. What is semantic caching? How is privacy protected?

Conceptually similar eligible first-turn public policy questions may reuse an answer using a high similarity threshold and qualifier checks. The cache is bounded to 128 entries with a 30-minute TTL. Private booking responses and live availability are excluded. Similar phrasing alone must not allow one user's booking data to be served to another.

Q116. What are guardrails, and can they completely prevent prompt injection?

Input screening, sensitive-data masking/aliasing, a restricted tool set, ownership checks and output validation reduce risk. They cannot prove that every malicious prompt is recognized. The stronger protection is that tools are read-only and enforce authorization outside the model. Model-generated instructions never grant extra privileges.

Q117. Can a user ask the AI to show another person's PNR?

The authenticated identity and service ownership checks remain authoritative. A guessed PNR or "ignore your instructions" prompt must not bypass them. A conversation ID is also not sufficient authority: it must belong to the current account.

Q118. Is output streamed token by token? Why structured output?

The backend uses WebFlux/SSE, but the implementation validates structured paragraph JSON before releasing content and builds cards from trusted service data. It should not be described as blindly forwarding every raw model token. BeanOutputConverter describes the expected answer shape; validation is still required because a model can produce malformed output.

Q119. Does SSE mean the whole application is reactive?

No. SSE is a streaming HTTP response mechanism; WebFlux is used in the chatbot path. Other services use blocking JPA/HTTP patterns. Reactive libraries do not make database work automatically nonblocking or remove external-provider latency.

Q120. Why might one question fail while the next succeeds?

The first may require a rate-limited Gemini call or failed embedding/tool request, while another uses cache or a direct account-query path. A 429 can mean a rate or quota limit, not necessarily that the entire daily allowance is exhausted. Check the service's failure category, model configuration and dependency status rather than displaying the same misleading message for every error.

Q121. How does the AI circuit breaker help?

Repeated provider failures open the Gemini breaker so calls fail quickly with a clear fallback instead of occupying resources indefinitely. It later probes recovery. Timeouts still bound individual requests. This does not increase free-tier quota or guarantee provider availability.

Q122. Why no Redis, Docker or external vector database?

Standalone portability was a project constraint. In-memory vector storage, memory and cache reduce setup for a local demonstration. The trade-off is restart loss and state separation across replicas. Shared persistent stores would be needed for robust horizontal scaling.

Q123. Did you train your own AI?

No. Railwize uses Gemini through its Developer API, adds project-specific retrieval, guardrails and authorized tools, and uses deterministic handling for some account intents. That is application integration and grounding, not custom model training.

13. Frontend and user experience questions

Q124. Why Angular? What are standalone components?

Angular provides TypeScript-based components, routing, forms and a structured ecosystem. Standalone components declare their dependencies directly and can be lazy-loaded without requiring a feature NgModule. Railwize separates core API/state helpers, pages and shared controls.

Q125. What are signals and RxJS used for?

Signals hold reactive UI state such as the current user or visible data; derived state updates when dependencies change. RxJS helps compose asynchronous work. They do not replace backend persistence or authorization. Explain their actual use in a selected component rather than claiming every feature uses the same state pattern.

Q126. How do frontend and backend connect?

Angular calls relative `/api` URLs. During development its proxy forwards to the gateway on 8080. In production the development proxy is not bundled: a reverse proxy must route `/api` and serve Angular's static files, with SPA fallback for deep links.

Q127. What is CORS? Is it access control?

CORS controls which browser origins may read cross-origin responses. It does not authorize a passenger, protect against curl or replace ownership/role checks. Swagger on different ports also has distinct browser storage origins.

Q128. Why validate both frontend and backend?

Frontend validation makes forms understandable and avoids predictable failed requests. Backend validation protects against modified clients, direct API calls and races. For example, the timetable form disables endpoint times, while schedule-service rejects invalid endpoint values independently.

Q129. How do you prevent data from a previous account appearing after logout?

Clear authentication and account-specific visible state, scope recent-search keys to stable account identity, cancel/reset relevant ongoing flows and refresh owner-scoped lists. Frontend tests cover authentication/logout races and storage behavior. The backend still checks ownership even if a browser displays stale content.

14. Persistence, reliability and testing

Q130. What are primary keys, foreign keys and unique constraints?

A primary key identifies a row. A foreign key enforces a relation within the database boundary. A unique constraint rejects duplicate values such as a PNR or logical booking key. Cross-service identity is usually a stored reference validated through APIs, not a JPA join across every service's database.

Q131. Why save snapshots of prices and passengers?

Historical tickets need the values accepted at booking time. A later fare edit or Master List update should not retroactively change the agreement or old passenger record. Email event snapshots likewise preserve the state being communicated even if a later cancellation occurs before delivery.

Q132. What are ACID and isolation?

Atomicity means local transaction changes commit together or roll back; consistency means invariants/constraints remain satisfied; isolation concerns concurrent transactions; durability means committed data survives normal persistence recovery. Booking transitions explicitly use READ_COMMITTED and pessimistic coordination. Isolation alone is not a promise that every distributed operation is serializable.

Q133. How are database changes managed?

Fresh local schemas can use the configured JPA schema behavior; repository SQL directories contain reviewed/manual migrations for existing data. Do not claim automatic Flyway/Liquibase rollout if it is not configured. Production would benefit from versioned migration automation, backups and tested upgrade procedures. MySQL DDL cannot be treated like a fully rollbackable business transaction.

Q134. Explain logging and request IDs.

SLF4J is the logging facade and Logback handles configured output/rotation. A request reference helps correlate a browser/API failure with service logs. Useful logs describe operation/failure type without exposing passwords, authorization headers, OTPs or provider secrets. A request ID is not automatically a full distributed tracing system such as OpenTelemetry.

Q135. What is JUnit, and what is Mockito?

JUnit runs tests and assertions. Mockito supplies controlled fake collaborators so a test can isolate business behavior. For example, mock a catalogue client to test schedule validation without starting train-service. The mock proves behavior against the test contract; it does not prove the real downstream system currently works.

Q136. Unit versus integration versus end-to-end tests?

Unit tests isolate a class/rule. Integration tests combine meaningful components such as Spring MVC, repositories and a test database. Full live end-to-end tests exercise the deployed browser-to-provider chain. Railwize has JUnit/Mockito, isolated H2-backed integrations, Jasmine logic tests and Playwright browser scenarios with controlled APIs/providers; these must not all be called live production E2E tests.

Q137. What are the most important tests to show?

`BookingLifecycleTest`: concurrent allocation, idempotency/fare tampering, nonoverlapping seat reuse, expiry/late capture, whole-booking promotion, FIFO fairness and owned history. `TrainRunCancellationTest`: dated cancellation behavior. `PaymentFlowIntegrationTest`: provider/payment/refund flows. `JourneyPlannerTest`: routes/timing/nearby logic. Frontend tests cover auth, timetable normalization, account-specific state, assistant selection and run-cancellation/help screens.

Q138. What is H2's limitation?

It provides a fast isolated database, but MySQL-mode syntax is not identical to actual MySQL locking, query planning or transaction behavior. Passing H2 tests does not prove production contention safety or performance. Real MySQL concurrency and failure tests should supplement them.

Q139. What are Jasmine and Playwright? Is Jest installed?

Jasmine is the chosen frontend unit/service test framework, run through the repository's TypeScript/Node harness. Playwright exercises browser scenarios. Do not claim Jest or a Karma browser runner merely because they are common Angular options. Also distinguish template/production build verification from unit tests.

Q140. Was SonarQube implemented?

No. It was explicitly deferred. Automated tests and compiler/build checks are present, but they are not a substitute for a SonarQube quality report. Do not invent coverage percentages, quality-gate outcomes or test counts without a fresh corresponding report.

Q141. What happens if one dependency is down?

The result depends on the path. Auth unavailability fails closed for protected access; catalogue outage prevents reliable search/quotes; PNR outage prevents a new reference; payment outage blocks checkout/reconciliation; email outage delays delivery without undoing tickets; Gemini outage affects AI responses. Microservices isolate responsibilities but do not eliminate dependency chains.

Q142. How would you scale or productionize it?

Measure bottlenecks first. Improve planner indexing and bounded catalogue retrieval, add proven MySQL concurrency/load tests, move auth/AI shared state to suitable persistent stores, add distributed tracing/metrics, automate schema migrations, harden credential storage and HTTPS, introduce sensible rate limiting, operational refund/dead-letter workflows and provider-compliance controls. Horizontal replicas alone cannot fix run-lock contention or process-local state.

Q143. What features are deliberately not claimed?

No live railway inventory, real payments, RAC, mixed passenger confirmation, partial passenger cancellation, guaranteed berth preference, atomic multi-train ticket, live delay prediction, protected connection, arbitrary schedule replacement, partial-route disruption, native XLSX, custom-trained ML model, permanently stored chatbot conversations, guaranteed exactly-once SMTP or SonarQube integration. General abuse throttling was deferred, though OTP controls exist.

15. Edge-case drill: predict the result before demonstrating

These are expected behaviors to explain and verify in the configured application, not a claim that every live case was rerun while writing this guide.

Evaluator's action	Expected response/reason
Select the same boarding/destination station	Reject; a journey needs distinct ordered stops.
Request a booking quote with a negative or zero passenger count	Reject; supported range is 1-6.
Send a list containing a null passenger	Reject cleanly rather than crash while reading fields.
Request more passengers than the entire class capacity	Reject; a group that can never fit should not enter WL.
Use lowercase/spaced station codes	Trim and normalize valid codes; reject malformed values.
Select a past date	Reject.
Select today after the boarding departure	Reject even though the date itself is allowed.
Select beyond two calendar months	Reject; use calendar-month arithmetic.
Book destination before source in the route	Reject the direction.
Create train without publishing a timetable	Not a bookable search result.
Enter source arrival or destination departure directly through API	Backend rejects it, regardless of disabled frontend fields.
Enter arrival after departure without changing day	Reject chronology; correct overnight day numbers.
Repeat/decrease cumulative distance or repeat a station	Reject the published route.
Choose Tuesday Day-2 boarding for a Monday-origin train	Check Monday's source operating day; this can be valid.
Cancel Tuesday boarding using Tuesday as source run date	Wrong run identity; select the actual original source date.
Double-click Book with the same key/body	Replay the same logical booking.
Reuse that key with changed passengers	Conflict; use a new key for a genuinely new booking.
Reduce the fare in the request JSON	Server quote mismatch; reject.
Two people book the last overlapping berth	Serialize allocation; no duplicate overlapping seat assignment.

Evaluator's action	Expected response/reason
Book A -> B then B -> C	Same berth can be reused across nonoverlapping segments.
Request a LOWER berth	Preference recorded; no guarantee in the current allocation algorithm.
Close checkout and let two minutes pass	Hold expires and capacity becomes available through lifecycle processing.
Receive capture after expiry/cancellation	Full-refund path; do not revive the ticket.
Pay a WL ticket	It remains WAITING until complete allocation is available.
Release one seat when oldest WL group needs two	It stays WL; later smaller group does not bypass it.
Remaining WL reaches its one-hour cutoff	Cancel whole booking and request full test refund.
Cancel a paid CNF ticket	Apply configured passenger fee, normally 20%.
Operator cancels its dated run	Zero-fee full refund for affected active paid bookings.
Repeat cancellation/webhook	Preserve already recorded decision; avoid duplicate financial effects.
Passenger attempts another account's ticket/PDF	Reject ownership violation.
Passenger manually opens an admin URL or API	Guard improves navigation; backend denies ADMIN operation.
Block an account with existing journeys	Revoke access; do not erase bookings or stop refunds.
Switch users after saving recent searches	Only the current account's list appears.
Request profile before first personal-details update	Return empty editable profile view for valid account.
Omit station coordinates	Exact routes still usable; missing-location nearby suggestions cannot be calculated.
Ask chatbot for someone else's PNR	Ownership checks remain enforced.
Stop Gemini/lose quota	Explain provider/dependency failure; core booking remains separate.
Stop SMTP	Retry delivery; booking/in-app record remains committed.
Import a calendar event and later cancel ticket	Imported event is not automatically removed.
Export over 10,000 rows	Ask for narrower filters; no silent truncation.

16. A strong 15-20 minute demonstration

Prepare accounts, provider credentials and data before the evaluation. Use a future date inside the current booking window, not a hardcoded expired screenshot date. The showcase import is 60 stations, 27 trains and 165 stops: 20 researched real service identities plus 7 fictional feature-test services. Fares/capacities are sample values, not official live tariffs.

Time	Demonstration	What to explain
0-2 min	Give the opening explanation and architecture.	Problem, stack, ownership and test-mode scope.
2-4 min	Register/verify or show prepared verified login; open profile/Master List.	Verified identity, login guard, stable ownership and reusable passenger details.
4-6 min	Direct search; inspect timetable and fare.	Cumulative distance, operating day, class capacity and server quote.
6-8 min	Search NZM -> BSB using RW901+RW902.	One interchange, 90-minute wait, per-leg prices, independent tickets.
8-9 min	Search CSMT -> ADI with nearby stations enabled.	Show actual BDTS/SBT stations and geographic distance disclaimer.
9-12 min	Book using saved passenger; complete test payment.	Hold, signature/provider verification, readable seat, PDF/email/in-app update.
12-14 min	Use RW907 JAT -> SVDK, capacity 2, to demonstrate paid WL promotion.	Two separate 1-passenger CNF bookings fill capacity; a third pays WL; cancel one CNF to release a seat.
14-16 min	Cancel another future source-date run from `/admin/train-runs`.	Preview/reason, all classes/segments, full refund, other dates unaffected.
16-18 min	Dashboard/users/bookings/reports/CSV.	Precise metric definitions and journey-date grouping.
18-20 min	Assistant policy and account questions; help and selected tests/code.	RAG vs live tools, read-only safety, service boundaries and evidence.

For WL preparation, choose a departure more than one hour away and a run with known empty test inventory. The second ticket and waitlisted ticket can be prepared beforehand if payment checkout would consume the entire presentation. Do not delete real user data to reset a demo. Use different future dates or dedicated demonstration accounts.

Recommended sample assistant questions:

- What happens if my waitlisted booking is not confirmed?

- Give my booked tickets.

- Which tickets have I cancelled?

- How many bookings do I have?

- When is my next journey?

- Find a train between two configured stations on a valid future date.

Backup plan: Have the Help page, existing tickets, a PDF, captured email examples and local tests ready if an external provider is unavailable. Label screenshots and pre-created examples honestly. Do not present a mocked provider response as proof of a live provider call.

17. Practice these complete explanations

Explain reliability in 45 seconds.

I do not assume that every distributed step succeeds together. Seat changes are protected by local database transactions and locks. Repeated booking requests use idempotency keys. Provider callbacks can repeat or disappear, so payments/refunds have reconciliation. Booking emails use a durable outbox and notification inbox. A failed email does not undo a ticket, and a late payment does not steal a seat from another booking: it follows the refund path.

Explain security in 45 seconds.

Accounts are verified, passwords are BCrypt-hashed and public users cannot select ADMIN. JWT requests also check server-side sessions so logout and blocking take effect. Backend endpoints enforce role and ownership, including PDFs, saved passengers and chatbot tools. Google sign-in uses state and a one-time code exchange. Provider secrets stay on the backend, payment verification is server-side, and errors/logs avoid exposing credentials. I still treat XSS hardening, production throttling and deployment security as areas for further operational work.

Explain uniqueness in 45 seconds.

The distinguishing problem I address is journey discovery when an exact direct train is unsuitable. Railwize checks an interchange or nearby endpoints and presents feasible scheduled alternatives with transfer time, fares and availability. It reduces manual comparison while clearly showing actual stations and independent ticket conditions. The planner is deterministic and testable; AI is an additional interface, not the authority that invents train routes or seat availability.

Explain the hardest implementation challenge.

Coordinating seat inventory with asynchronous payment was the hardest part. A seat can be held while the user pays, but the payment may arrive late or twice. At the same time, another user may book, a waitlisted booking may be promoted or an administrator may cancel the run. I addressed this with explicit states, server deadlines, run/class locking, idempotency and compensating refunds instead of trusting one browser callback.

Explain a bug you solved.

Choose one that you understand and can show:

- Schedule form sent source arrival and destination departure, violating the API's route contract. The UI now disables/normalizes those fields while retaining server validation.

- `/me` assumed every auth account already had a profile row. The lookup now supports a valid account before profile initialization.

- Recent searches were browser-global. They now use stable account-specific storage and clear visible state on account changes.

- Google redirect mismatch came from a callback configuration mismatch; backend Google callback and Angular completion URLs serve different purposes.

- Treating every AI failure as exhausted quota obscured different provider/tool/embedding problems; status categories and direct account-question paths make the behavior more accurate.

Do not say "I fixed it by removing validation" or "the frontend guard secures the endpoint." Explain the contract and the evidence.

18. Difficult follow-up answers you should not improvise

Challenge	Defensible answer
Why not Redis locks?	Database locks protect the database invariant in the current topology without another dependency. Distributed locks introduce their own lease/failure correctness problems.
Why pessimistic instead of optimistic locking?	Hot inventory needs a single authoritative allocation decision. Optimistic retries could work but require careful whole-operation retry/idempotency; pessimistic locking is simpler here, with throughput costs.
Does consistent lock ordering eliminate every deadlock?	It reduces a major cause; it is not proof that no query or multi-key background transaction can ever deadlock. Real database tests and retry/error handling still matter.
Why might some seats stay empty while WL exists?	Strict whole-booking FIFO can block later groups behind a larger or different-segment head booking. This is a fairness policy, not a maximum-utilization optimizer.
Can cached availability be used to confirm?	No. Availability is a snapshot and final allocation must recheck authoritative state under coordination.
Does outbox guarantee email content is current?	It preserves the event snapshot. An old confirmation may arrive after a later cancellation; events and current ticket state serve different purposes.
Is schema-per-service the same as data isolation?	It creates ownership boundaries, but permissions, credentials and application queries must also respect them; one shared MySQL host is still a common infrastructure dependency.
Would two chatbot replicas share memory?	No, current state is process-local. Shared persistence/session affinity or a redesign is needed.
Is the free Gemini tier unlimited?	No. Model availability/rate/quota conditions belong to the provider account and can change; the application must handle limits.
Why is a booking list sometimes available when AI is down?	Some account intents use authorized deterministic service calls without the LLM.
Is a generated PDF an invoice compliant with every tax rule?	No. It is a project ticket/fare document; a full regulated tax-invoice system is not claimed.
Is `READ_COMMITTED` enough for inventory?	The isolation setting works together with explicit shared locking and invariant checks, not by itself.
How do you prove no overselling?	Show overlapping-seat logic, transaction/lock boundaries and concurrency tests; then qualify that real MySQL load/failure tests are needed for stronger operational evidence.
Did all automated tests run today?	This guide is source-based; show actual current test reports if you rerun them. Never invent a pass or coverage number.

19. How to answer an unfamiliar question

Use this structure:

1. **State what the current code does.** Be specific about the relevant service and rule.
2. **Explain the reason.** Identify the invariant or user problem.
3. **Give a small example.** Numbers or two concurrent requests make the explanation concrete.
4. **Point to evidence.** A method, entity constraint or named test.
5. **State the boundary.** Distinguish implemented behavior from a proposed improvement.

If the answer is not known, say: "I would verify that in this class/test before claiming it. The intended rule is X, and I would test it with Y." That is stronger than inventing a confident guarantee.

Claims to avoid: "JWT is encrypted," "all payments immediately confirm seats," "microservices automatically scale," "the planner is AI/ML," "emails are exactly once," "the PDF cannot be forged," "all tests are live E2E," "our reports equal bank revenue," "all real-world berth layouts are supported," or "the system has no remaining bugs."

20. A focused revision plan

First session - explain the product. Practice the opening answer, list passenger/admin capabilities and perform a normal booking. Learn the service responsibilities rather than reciting all ports first.

Second session - defend the hard flows. Draw booking/payment/refund states; work through last-seat concurrency, two-minute expiry, paid WL FIFO, segment reuse and dated cancellation. Calculate one fare and explain one overnight run.

Third session - demonstrate and locate code. Run the unique planner examples, assistant and admin reports; open the classes in the following index and the matching tests. Practice what you will say when a provider is unavailable.

Before presenting, verify the configured app and data, choose valid future dates, confirm admin access and keep the terminal/logs available. Do not expose `.env`, API keys, session tokens or personal test-user details on a projected screen.

21. Implementation evidence index

Each entry below links to the project source in this repository. Begin with the implementation and then show the matching test where available. The relative links work on GitHub and in a downloaded copy of the repository.

Feature / evidence	Repository source
Registration, verification and credentials	backend/auth-service/src/main/java/com/railway/authservice/service/AuthService.java
Google callback and exchange	backend/auth-service/src/main/java/com/railway/authservice/controller/GoogleOAuthController.java
Sessions and logout	backend/auth-service/src/main/java/com/railway/authservice/service/SessionService.java
Admin invitations	backend/auth-service/src/main/java/com/railway/authservice/service/AdminInvitationService.java
One-time username change	backend/auth-service/src/main/java/com/railway/authservice/service/UsernameService.java
Admin user management	backend/auth-service/src/main/java/com/railway/authservice/service/AdminUserService.java
Profile initialization and ownership	backend/user-service/src/main/java/com/railway/userservice/service/UserProfileService.java
Saved passengers	backend/user-service/src/main/java/com/railway/userservice/service/SavedPassengerService.java
Train and class configuration	backend/train-service/src/main/java/com/railway/trainservice/service/TrainService.java
Stations and coordinates	backend/train-service/src/main/java/com/railway/trainservice/service/StationService.java
Timetable validation and publication	backend/schedule-service/src/main/java/com/railway/scheduleservice/service/ScheduleService.java
Fare, date and operating-day rules	backend/reservation-service/src/main/java/com/railway/reservationservice/service/JourneyService.java
Connections, nearby search and ranking	backend/reservation-service/src/main/java/com/railway/reservationservice/service/JourneyPlanner.java
Idempotency and quote comparison	backend/reservation-service/src/main/java/com/railway/reservationservice/service/ReservationService.java
Allocation, holds, paid WL, cancellation and transitions	backend/reservation-service/src/main/java/com/railway/reservationservice/service/BookingEngine.java
Pessimistic inventory lock	backend/reservation-service/src/main/java/com/railway/reservationservice/repository/SeatInventoryRepository.java
Segment overlap query	backend/reservation-service/src/main/java/com/railway/reservationservice/repository/ReservationRepository.java

Feature / evidence	Repository source
Initial inventory and run lock row	backend/reservation-service/src/main/java/com/railway/reservationservice/service/InventoryInitializer.java
Dated cancellation validation	backend/reservation-service/src/main/java/com/railway/reservationservice/service/RunCancellationService.java
Readable berth mapping	backend/reservation-service/src/main/java/com/railway/reservationservice/service/SeatLayouts.java
PDF generation	backend/reservation-service/src/main/java/com/railway/reservationservice/service/TicketPdfService.java
In-app notification records	backend/reservation-service/src/main/java/com/railway/reservationservice/service/NotificationCenter.java
Expiry and event jobs	backend/reservation-service/src/main/java/com/railway/reservationservice/service/ReservationJobs.java
Booking-event HTTP handoff	backend/reservation-service/src/main/java/com/railway/reservationservice/service/NotificationRelay.java
Booking reports and CSV	backend/reservation-service/src/main/java/com/railway/reservationservice/service/AdminBookingService.java
PNR circuit breaker	backend/reservation-service/src/main/java/com/railway/reservationservice/service/PnrGateway.java
PNR allocation algorithm	backend/pnr-service/src/main/java/com/railway/pnrservice/service/PnrService.java
PNR database uniqueness	backend/pnr-service/src/main/java/com/railway/pnrservice/entity/PnrRecord.java
Payment verification and refund orchestration	backend/payment-service/src/main/java/com/railway/paymentservice/service/PaymentService.java
Notification deduplication/retries	backend/notification-service/src/main/java/com/railway/notificationservice/service/NotificationInbox.java
Booking email rendering/delivery	backend/notification-service/src/main/java/com/railway/notificationservice/service/EmailService.java
Consistent reservation API errors	backend/reservation-service/src/main/java/com/railway/reservationservice/exception/ApiErrors.java
Resource authentication filter	backend/reservation-service/src/main/java/com/railway/reservationservice/config/JwtAuthFilter.java
RAG loading/retrieval	backend/chatbot-service/src/main/java/com/railway/chatbotservice/service/KnowledgeBase.java
Assistant live tools	backend/chatbot-service/src/main/java/com/railway/chatbotservice/service/RailwayTools.java
Direct account-question handling	backend/chatbot-service/src/main/java/com/railway/chatbotservice/service/AccountQuestion.java

Feature / evidence	Repository source
Conversation ownership/memory	backend/chatbot-service/src/main/java/com/railway/chatbotservice/service/Conversations.java
Semantic policy cache	backend/chatbot-service/src/main/java/com/railway/chatbotservice/service/SemanticCache.java
Guardrail advisor	backend/chatbot-service/src/main/java/com/railway/chatbotservice/service/GuardrailAdvisor.java
Assistant flow and resilience	backend/chatbot-service/src/main/java/com/railway/chatbotservice/service/ChatService.java
Structured-answer decoder	backend/chatbot-service/src/main/java/com/railway/chatbotservice/service/AnswerDecoder.java
Frontend routes	frontend/src/app/routes.ts
Frontend auth/API/guards	frontend/src/app/core/api.ts
Source/destination time normalization	frontend/src/app/core/timetable.ts
Passenger selection and booking	frontend/src/app/pages/book.ts
Account-scoped recent searches	frontend/src/app/core/recent-searches.ts
Nearby/connecting journey UI	frontend/src/app/shared/journey-alternatives.ts
Ticket and cancellation UI	frontend/src/app/pages/ticket.ts
Calendar generation	frontend/src/app/core/calendar.ts
Notification polling	frontend/src/app/core/notifications.ts
Assistant shared state	frontend/src/app/core/chat-session.ts
Assistant SSE parser	frontend/src/app/core/chat-stream.ts
Assistant page/panel UI	frontend/src/app/pages/chat.ts
Admin insights	frontend/src/app/pages/admin-insights.ts
Admin users/bookings	frontend/src/app/pages/admin-management.ts
Dated cancellation UI	frontend/src/app/pages/admin-runs.ts
Help centre	frontend/src/app/pages/help.ts
Booking concurrency/lifecycle tests	backend/reservation-service/src/test/java/com/railway/reservationservice/service/BookingLifecycleTest.java
Journey planner tests	backend/reservation-service/src/test/java/com/railway/reservationservice/service/JourneyPlannerTest.java
Dated cancellation tests	backend/reservation-service/src/test/java/com/railway/reservationservice/service/TrainRunCancellationTest.java

Feature / evidence	Repository source
Payment integration tests	backend/payment-service/src/test/java/com/railway/paymentservice/service/PaymentFlowIntegrationTest.java
Profile integration tests	backend/user-service/src/test/java/com/railway/userservice/controller/ProfileLifecycleIntegrationTest.java
Frontend auth tests	frontend/tests/jasmine/auth.spec.ts
Frontend timetable tests	frontend/tests/jasmine/timetable.spec.ts
Browser run-cancellation/help tests	frontend/tests/browser/train-runs-help.spec.ts
India demonstration dataset guide	Railwize-india-data/INDIA-DATA-GUIDE.txt

22. Latest features and deeper technical follow-ups

Q144. Why did MMCT to DLI miss a useful MMCT to NDLS alternative?

Exact station search correctly distinguishes Delhi Junction (DLI) from New Delhi (NDLS). The nearby planner must expand both endpoints, not just the origin, and must have usable coordinates for each station. The updated implementation examines nearby direct routes before connections and pages through a bounded catalogue. A busy earlier interchange should not consume the work budget before a useful destination alternative is checked. This is a candidate-discovery problem, not a fare-formula problem.

Q145. How would you demonstrate that correction?

Use a future valid date, an operating MMCT to NDLS service, a published timetable and usable DLI/NDLS coordinates. Search MMCT to DLI with nearby stations enabled and a sufficient radius. Inspect the result's actual arrival station: New Delhi (NDLS). Repeat with a nearby origin as well, then disable nearby search to establish the difference. Compare against an exact MMCT to NDLS search. Booking must retain the actual endpoints, not replace NDLS with DLI.

Q146. What does the station-coordinate repair SQL do?

REPAIR-STATION-COORDINATES.sql repairs missing or invalid location pairs for recognized station codes using the supplied dataset's approximate coordinates. It preserves valid nonzero pairs and reports unrecognized/custom stations. It does not invent train schedules, modify booking history or fix an incorrect but valid-looking coordinate. Back up the catalogue first and follow the script instructions. Uploading the file to GitHub does not execute it against MySQL.

Q147. Why show Surat (ST) instead of only ST?

Station codes are stable machine identifiers but poor explanations for unfamiliar passengers. The planner returns a station-name map and the UI combines human-readable names with codes. Actual train endpoints and interchange names remain visible. A name lookup failure can fall back to the code with a notice; the application must not guess a station identity.

Q148. How does the support-ticket workflow work?

A signed-in user creates a categorized request with a subject, message and optional PNR belonging to that account. Admins use the support inbox to search requests, reply and change status. Replies to resolved tickets reopen the request. Email and in-app updates link back to the protected conversation. Creating a support request does not automatically cancel a train, issue a refund or guarantee a response deadline.

Q149. How do support requests handle retries and concurrent admin changes?

The creation request includes an idempotency reference. The server derives an owner-scoped request identity and compares a request-content hash on retries: identical requests return the original case, while changed content with the same reference conflicts. Conversation mutations check the current ticket version so a stale admin action cannot silently overwrite a newer update. See SupportService.java and SupportTicket.java, and demonstrate a refresh after a conflict.

Q150. How is the new visual theme implemented without changing booking rules?

The Angular application loads shared theme.css after the base styles. Reusable colour tokens and component styling keep the passenger pages, admin workspace, help, assistant and checkout consistent. Email templates use inline styles and bundled logo images because email clients do not load Angular CSS. The navy/cobalt/cyan theme is presentation; backend fare, ownership and allocation rules remain authoritative. Build checks and browser scenarios verify that styling has not hidden required interactions.

Q151. Where do Java collections help in this project?

The planner uses maps to index routes by departure station and retain candidate plans by identifier. Sets deduplicate station codes and route validation rejects repeated stations. Lists preserve ordered timetable stops. A HashMap typically offers average constant-time lookup, but that does not make the overall planner constant-time: enumerating legs, classes, dates and destinations still costs work and remote quotes dominate latency. Choose a collection for the required ordering, uniqueness and lookup behavior.

Q152. What is optimistic versus pessimistic locking?

Pessimistic locking acquires database coordination before deciding an allocation; another writer waits or fails. Optimistic locking detects that a version changed and rejects a stale write. Seat allocation needs carefully ordered run/inventory locks to protect a scarce resource. Support updates use version checks to prevent stale decisions. Neither approach creates a transaction across payment-provider HTTP calls. Handle conflicts and deadlocks with bounded, safe retries only where operations are idempotent.

Q153. Which indexes would you investigate if booking history became slow?

Start from the actual query and EXPLAIN plan. Owner identity, journey date, status, train/run identity and ordering columns are candidates for suitable composite indexes. Equality predicates generally determine an index prefix, followed by range or order requirements. Indexes speed reads but add storage and write cost. Do not claim a particular index exists without opening the entity/schema; measure with realistic data and preserve pagination limits.

Q154. How would you test a last-seat race convincingly?

Create one available berth for a known train/run/class and overlapping route segments. Start two independent transactions together using a barrier. Assert that at most one booking receives that berth for the overlapping interval, and verify the database state rather than only HTTP responses. Test nonoverlapping segments separately, repeat on MySQL, and include expiry and cancellation interleavings. A sequential test or a mocked repository cannot prove database concurrency behavior.

Q155. What would you monitor in a deployed version?

Search/quote latency, booking conflicts, hold expiry, payment reconciliation lag, refund backlog, notification retry age, dependency timeouts and authorization failures. Correlate failures using request references without logging tokens or OTPs. These are proposed operational metrics; the current project has logging and request IDs, not a claimed complete metrics/tracing deployment. Define alerts against measured baselines rather than arbitrary impressive numbers.

Q156. How do you explain an implementation limitation confidently?

Say what the current code guarantees, show its boundary, and explain the next step. For example: "I coordinate each booking locally and recover provider failures with reconciliation; I do not claim an atomic transaction across two train legs. An integrated multi-leg checkout would need compensation, recovery and clear passenger consent if the second leg fails." This is stronger than claiming every operation is instantaneous, perfectly secure or exactly once.

Additional code pointers

Feature	Repository-relative location
Support workflow and version checks	backend/reservation-service/src/main/java/com/railway/reservationservice/service/SupportService.java
Support ticket persistence	backend/reservation-service/src/main/java/com/railway/reservationservice/entity/SupportTicket.java
Support frontend	frontend/src/app/pages/support.ts
Shared visual palette	frontend/src/theme.css
Account email shell	backend/auth-service/src/main/java/com/railway/authservice/service/RailwizeMail.java
Booking/support email shell	backend/notification-service/src/main/java/com/railway/notificationservice/service/RailwizeMail.java
Coordinate repair	Railwize-india-data/REPAIR-STATION-COORDINATES.sql

Final preparation checklist

- Practice Q18, Q25, Q43, Q51, Q62-67, Q75, Q84-92 and Q113-118 aloud.
- Open the implementation files before the interview; know the transaction and authorization boundaries.
- Use future dates and test-mode credentials for demonstrations. Keep secrets out of screenshots.
- Distinguish an automated mock-backed test from a live provider demonstration.
- Explain the nearby planner with MMCT/DLI/NDLS and name every actual interchange station.
- State that SonarQube is deferred and that no real railway inventory or money is involved.